

Manual de Estrutura de Projeto e Submissão no Kaggle

Este guia orienta como estruturar o código do seu agente, testá-lo localmente e submetê-lo à plataforma Kaggle Environments.

1. Estrutura Mínima do Projeto

Para manter o desenvolvimento simples, modular e sem complicações com imports, todo o código do agente fica concentrado na pasta `src/`:

```
meu-projeto/
├── pyproject.toml      # Gerenciador de dependências (uv)
├── src/                # Código-fonte do agente
│   ├── main.py        # Entrypoint da submissão (contém a função agent)
│   ├── agente1.py     # Lógica e heurística do seu agente
│   ├── modulo_auxiliar.py  # (Opcional) Classes auxiliares, parsers ou utilitários
│   └── constantes.py  # (Opcional) Constantes e parâmetros
Nota sobre imports: Como todos os arquivos estão juntos dentro de src/, os imports entre eles são diretos (ex: from agente1 import meu_agente as agent), dispensando manipulações de sys.path ou arquivos __init__.py.
```

2. O Arquivo de Entrada (`src/main.py`)

A plataforma Kaggle exige que o arquivo principal se chame `main.py` e exponha uma função chamável chamada `agent`:

```
# src/main.py

# Importa a função que contém a lógica do seu agente
from agente1 import meu_agente as agent

if __name__ == "__main__":
    import webbrowser
    from kaggle_environments import make

    OPPONENT = "random" # Adversário de teste: "random", "pass" ou "starter"
```

```
EPISODE_STEPS = 720 # Duração da simulação em turnos
SAVE_HTML = True    # Gera replay visual no navegador

print(f"Iniciando teste local contra '{OPPONENT}' ({EPISODE_STEPS} turnos)...")
env = make("kaggriculture", configuration={"episodeSteps": EPISODE_STEPS, debug=True)
env.run([agent, OPPONENT])

p0, p1 = env.steps[-1]
print("-" * 45)
print(f"Seu Agente (P0): ${p0.reward:,.2f} | Status: {p0.status}")
print(f"Adversário (P1): ${p1.reward:,.2f} | Status: {p1.status}")
print("-" * 45)

if SAVE_HTML:
    with open("resultado.html", "w", encoding="utf-8") as f:
        f.write(env.render(mode="html", width=1000, height=750))
    webbrowser.open("resultado.html")
```

3. Configuração do Ambiente e Credenciais

3.1 Instalar dependências

Usando o gerenciador `uv`:

```
uv add kaggle-environments kaggle
uv sync
```

3.2 Autenticação no Kaggle

Para submeter diretamente via terminal:

1. Acesse kaggle.com/settings/api.
2. Clique em **"Create New Token"** para baixar o arquivo `kaggle.json`.
3. Salve o arquivo na sua máquina e ajuste as permissões:

```
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json
```

Atenção: Certifique-se de ter clicado em **"Join Competition"** na página oficial da competição para aceitar suas regras antes da primeira submissão.

4. Teste Local

Antes de enviar, valide o comportamento do seu agente na sua máquina:

```
uv run python src/main.py
```

O comando executará uma partida completa, exibirá os saldos no terminal e abrirá o replay interativo no navegador.

5. Empacotamento e Submissão

O Kaggle exige que o arquivo `main.py` esteja localizado na **raiz** do arquivo compactado de submissão.

Passo 1: Gerar o pacote `.tar.gz`

Execute a partir da raiz do repositório:

```
tar --exclude='*__pycache__*' -C src -czf submission.tar.gz .
```

(A flag `-C src` entra na pasta e o `.` empacota automaticamente **todos** os arquivos presentes nela direto na raiz do arquivo final, enquanto `--exclude` ignora pastas de cache temporário).

Passo 2: Enviar ao Kaggle

```
uv run kaggle competitions submit kaggriculture -f submission.tar.gz -m "Descrição da versão"
```

6. Acompanhamento

Verifique o processamento da submissão na fila:

```
uv run kaggle competitions submissions kaggriculture
```

- Status **PENDING**: O agente está na fila de avaliação.
- Status **COMPLETE**: O agente passou nos testes e está ativo no ranking (*leaderboard*).
- Status **ERROR**: Ocorreu alguma falha de execução ou timeout durante a validação.